

Shakespeare Text Generator: Character-Level LSTM

Scaler School of Technology | Deep Learning I | Week 8 — Session 3/5 | Post-Lecture Review & Practice



Key Takeaways

1 LSTMs solve vanishing gradients through gating

Three learnable gates control information flow. The **cell state** c_t acts as a gradient highway: $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ where f_t (forget), i_t (input), o_t (output) are sigmoid-gated. The additive update prevents gradients from vanishing over hundreds of timesteps, unlike vanilla RNNs.

2 Character-level modeling learns hierarchical structure

Starting from individual characters, the LSTM learns: characters $\rightarrow$ spelling $\rightarrow$ words $\rightarrow$ grammar $\rightarrow$ style. No tokenizer is needed, the vocabulary is tiny (~ 65 unique chars), and the model can generate any Unicode sequence including code, markup, and invented words.

3 Temperature controls generation diversity

During sampling, logits are divided by temperature T before softmax: $p_i \propto \exp(z_i/T)$. Low $T < 1$ sharpens the distribution (conservative, repetitive). High $T > 1$ flattens it (creative, incoherent). The sweet spot $T \approx 0.8$ balances coherence and creativity — a fundamental trade-off in all generative models.

Test Your Understanding

Try answering these without looking at the slides. Answers are on page 3.

- Q1.** What is the role of the forget gate in an LSTM? Give a concrete example of what information it might “forget” when modeling Shakespeare.
- Q2.** Why do we clip gradients to a maximum norm of 5 during LSTM training? What problem does gradient clipping address?
- Q3.** When generating text, we sample from the probability distribution rather than always taking the argmax. Why? What goes wrong with greedy decoding?
- Q4.** A character-level model generates “HAMLE” when prompted with “HAM”. What does this reveal about the model’s local pattern-matching ability and its limitations for long-range planning?
- Q5.** Compare word-level vs. character-level language modeling: give two advantages and two disadvantages of the character-level approach.

Coding Exercises

Work through these in a Colab notebook with GPU enabled (Runtime > Change runtime type > T4 GPU).

Exercise 1: Train CharLSTM and Track Learning Progress

Train the full CharLSTM (Embedding $\rightarrow$ 2-layer LSTM $\rightarrow$ Linear) on Tiny Shakespeare for 20 epochs. Record 200-character generated samples at epochs 1, 5, 10, and 20 using the same seed “HAMLET:” and temperature $T = 0.8$. Plot the training loss curve. *Expected output:* loss curve showing decreasing perplexity + 4 text samples demonstrating progression from gibberish to coherent Shakespearean text.

Exercise 2: Temperature Ablation Study

After training, generate 200 characters with temperatures $T \in \{0.1, 0.3, 0.5, 0.8, 1.0, 1.5, 2.0\}$ using the same seed “To be or not to be,”. For each output, count the number of unique bigrams (pairs of consecutive characters). Plot unique bigrams vs. temperature. *Expected output:* monotonically increasing diversity curve + 3 representative text samples at $T = 0.2$, $T = 0.8$, $T = 1.5$.

Exercise 3: RNN vs LSTM Head-to-Head

Implement a vanilla RNN by replacing `nn.LSTM` with `nn.RNN` (same hidden size=256, 2 layers). Train both models from scratch for 20 epochs on the same data with identical hyperparameters. Plot both training loss curves on the same axes. Generate 200 characters from each model and compare quality. *Expected output:* comparison loss plot (LSTM should achieve lower final loss) + side-by-side text samples.

Challenge Problem

Train a Python Code Generator:

1. Download a Python source file corpus: concatenate all `.py` files from any open-source GitHub repo (e.g., `requests` library, ~500KB)
2. Preprocess: keep only printable ASCII chars; build char vocabulary
3. Train the same CharLSTM architecture for 15 epochs (same hyperparameters)
4. Generate code starting with the seed “`def train()`” — does the model produce syntactically valid Python?
5. Compute the percentage of 200-character snippets that parse without errors using `ast.parse()`

Bonus: Compare code quality when training on 50KB vs 500KB of Python source code. How does data volume affect syntactic validity?

Further Reading

- **Original LSTM:** Hochreiter & Schmidhuber (1997), “*Long Short-Term Memory*” — surprisingly readable; the cell state and gate formulas are directly implemented in `nn.LSTM`.
- **Karpathy’s Blog:** Karpathy (2015), “*The Unreasonable Effectiveness of Recurrent Neural Networks*” — char-level models on Shakespeare, LaTeX, C code, Wikipedia. The motivating piece for this session.
- **First Neural LM:** Bengio et al. (2003), “*A Neural Probabilistic Language Model*” — introduced word embeddings; foundational reading before studying GPT.
- **Advanced LSTM Regularization:** Merity et al. (2018), “*Regularizing and Optimizing LSTM Language Models*” — DropConnect, AR/TAR regularization, achieving SOTA on Penn Treebank.

Online Resources & Tools

- **Karpathy char-rnn** — original Lua implementation
- **PyTorch nn.LSTM** — official documentation
- **Lilian Weng’s LSTM blog** — detailed explainer
- **HuggingFace tokenizers** — word-level alternative
- **Weights & Biases** — track experiment runs
- **nanoGPT** — Karpathy’s minimal GPT (next step)

Fill the Gaps — Prerequisite Refreshers

Felt lost on a specific concept? These resources will help you catch up.

- **Confused about embeddings vs one-hot?** `nn.Embedding` learns dense d -dim representations. It is mathematically equivalent to a linear layer on a one-hot vector but computed efficiently as a lookup table.
- **Unsure why we pad sequences?** Batches require equal-length tensors. Padding fills short sequences with a PAD token; use `ignore_index` in `CrossEntropyLoss` to ignore PAD positions during loss computation.
- **Shaky on cross-entropy loss?** Revisit Week 2 Session 3 — for a vocabulary of size V , `CrossEntropyLoss` takes raw logits (not softmax’d) and computes $-\log p_{\text{true}}$.
- **Forgot about gradient clipping?** `torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5)` rescales all gradients if their global norm exceeds 5, preventing explosive updates.

Answers — Test Your Understanding

- A1.** The forget gate $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$ determines how much of the previous cell state to retain. In Shakespeare modeling, when a new character’s speech begins (e.g., “HAMLET:” appears), the model should forget the previous speaker’s context — the forget gate learns to activate and clear stored speaker-specific information.
- A2.** In deep unrolled RNNs, gradients multiply through W_{hh} at each timestep. If $\|W_{hh}\| > 1$, gradients explode exponentially with sequence length. Clipping constrains the gradient’s global norm, preventing a single catastrophically large update that would destabilize training. It does not prevent vanishing gradients (that requires LSTM’s architecture).
- A3.** Greedy decoding (always taking `argmax`) leads to degenerate repetitive text — the model gets stuck in loops like “the the the the”. Sampling from the distribution introduces randomness proportional to model confidence, producing varied and natural sequences. Temperature lets us tune how much randomness we inject.
- A4.** The model correctly continues “HAMLET” after “HAM”, demonstrating it learned local character n -gram statistics. The limitation is that it cannot “plan” the full word before generating the first character — it predicts one character at a time based on context, with no lookahead.
- A5. Character-level advantages:** (1) No tokenizer needed; works on any text including code, math, and URLs. (2) Fixed, tiny vocabulary (~ 65 chars) vs. 50K+ word tokens. **Disadvantages:** (1) Must model longer sequences to capture word-level patterns (100 chars vs. 20 words). (2) Slower training and inference per “semantic unit” generated.

Deep Dive: LSTM Gate Equations

Given input x_t and previous hidden state h_{t-1} , the LSTM computes:

$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$	(forget gate: what to discard from c_{t-1})
$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$	(input gate: what new info to write)
$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$	(candidate cell: new content)
$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$	(cell state update: additive!)
$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$	(output gate: what to expose)
$h_t = o_t \odot \tanh(c_t)$	(hidden state output)

Key insight: c_t uses an **additive** update $c_t = f_t c_{t-1} + i_t \tilde{c}_t$. Additive updates allow gradients to flow back through time without repeated multiplication — this is the “gradient highway”.

Deep Dive: Why Vanilla RNNs Fail

In a vanilla RNN, the hidden state update is $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$.

The gradient from loss at time T to hidden state at time 1 is:

$$\frac{\partial h_T}{\partial h_1} = \prod_{t=1}^T W_{hh}^\top \cdot \text{diag}(\tanh'(z_t))$$

If the spectral radius $\rho(W_{hh}) < 1$: gradients **vanish** exponentially ($\sim \rho^T \rightarrow 0$). If $\rho > 1$: gradients **explode**. For $T = 100$ timesteps (our sequence length), even $\rho = 0.9$ gives $0.9^{100} \approx 2.6 \times 10^{-5}$ — effectively zero gradient at the start.

LSTMs replace this multiplicative chain with an additive cell state update, so $\frac{\partial c_T}{\partial c_1} \approx \prod f_t$ (product of forget gate values near 1 when the context is worth preserving).

Deep Dive: Embedding Layer vs One-Hot Encoding

One-Hot Encoding:

- Sparse: V -dim vector, 1 non-zero
- No parameters for lookup
- Each character treated identically
- No learned similarity structure

Learned Embedding:

- Dense: d -dim vector (e.g., $d = 128$)
- $V \times d$ learnable parameters
- Similar characters $\rightarrow$ similar vectors
- Computationally equivalent to $W \cdot \mathbf{e}_i$

`nn.Embedding(vocab_size, 128)` is a lookup table: character index i maps to row i of the weight matrix, updated during backpropagation. This allows “e” and “E” to end up nearby in embedding space, as the model learns their functional similarity.

Perplexity: The Standard LM Evaluation Metric

Perplexity measures how “surprised” the model is by the test data:

$$\text{PPL} = \exp\left(-\frac{1}{N} \sum_{i=1}^N \log p(x_i | x_{<i})\right) = e^H$$

where H is the cross-entropy loss. **Lower perplexity = better model.**

Interpretable: $\text{PPL} = k$ means the model is as uncertain as if uniformly choosing among k options at each step. A model with $\text{PPL} = 4$ assigns the correct character as one of 4 roughly equally likely choices. Random uniform prediction over 65 chars gives $\text{PPL} = 65$.

Implement perplexity tracking: after each epoch compute `pp1 = math.exp(val_loss)` and plot alongside the loss curve.

Advanced Sampling: Top-k and Nucleus (Top-p)

Three sampling strategies, in order of increasing sophistication:

1. **Temperature sampling:** Scale all logits by $1/T$, then sample. Simple but may include very unlikely tokens at high T .
2. **Top-k sampling:** Zero out all logits except the top k (e.g., $k = 40$), renormalize, then sample. Prevents sampling from the long tail.
3. **Nucleus (top-p) sampling:** Find the smallest set of tokens whose cumulative probability exceeds p (e.g., $p = 0.9$). Sample from only those tokens. Adapts dynamically: sharp distributions $\rightarrow$ small set; flat distributions $\rightarrow$ large set.

Implement all three and compare generated text quality on the same seed and model. Modern LLMs (GPT-4) use top-p sampling by default.

Scaling Laws & the Path from LSTM to Transformer

The CharLSTM you built has $\sim 2\text{M}$ parameters and trains in minutes. Compare:

Model	Parameters	Data	Architecture
CharLSTM (this session)	$\sim 2\text{M}$	1.1MB	2-layer LSTM
Karpathy's char-rnn	$\sim 10\text{M}$	1MB	3-layer LSTM
GPT-1 (2018)	117M	BookCorpus	12-layer Transformer
GPT-2 (2019)	1.5B	WebText	48-layer Transformer
GPT-3 (2020)	175B	570GB	96-layer Transformer

The architecture change (LSTM $\rightarrow$ Transformer) was the key breakthrough: Transformers process all tokens in parallel, enabling training on vastly more data. The training loop, loss function, and autoregressive generation paradigm you used today are identical to GPT.

RNN Variants: Architecture Comparison

Architecture	Gates	Parameters	Gradient Flow	Best For
Vanilla RNN	None	Fewest	Vanishes >20 steps	Very short sequences
LSTM	3 (f, i, o)	Most	Cell state highway	Long sequences, default choice
GRU	2 (r, z)	Less than LSTM	Good	Faster training, comparable quality
Bi-LSTM	3 + reverse	2× LSTM	Excellent	Encoding (not generation)
Transformer	N/A	Varies	Direct (attention)	Long-range, parallelizable

Temperature Sampling Quick Reference

Temperature	Distribution Shape	Generated Text Characteristics
$T = 0$	One-hot (greedy)	Fully deterministic; often loops
$T = 0.2$	Very peaked	Conservative; repetitive but coherent
$T = 0.5$	Moderately peaked	Safe; grammatical but predictable
$T = 0.8$	Slightly spread	Recommended — balanced quality/creativity
$T = 1.0$	Raw model distribution	Unmodified sampling
$T = 1.5$	Flattened	Creative but often incoherent
$T = 2.0$	Near-uniform	Largely random character sequences

Code snippet:

```
def sample(model, seed, length, temperature=0.8):
    model.eval()
    chars = [char2idx[c] for c in seed]
    with torch.no_grad():
        for _ in range(length):
            x = torch.tensor(chars[-100:]).unsqueeze(0)
            logits, _ = model(x)
            logits = logits[0, -1] / temperature # apply temperature
            probs = torch.softmax(logits, dim=-1)
            next_char = torch.multinomial(probs, 1).item()
            chars.append(next_char)
    return ''.join([idx2char[i] for i in chars])
```

Training Debugging Checklist

Symptom	Likely Cause	Fix
Loss constant from epoch 1	Learning rate too small or zero	Check optimizer initialization; try LR = 0.002
Loss explodes (NaN)	Gradients exploding	Add <code>clip_grad_norm_(model.parameters(), 5)</code>
Only spaces/newlines generated	Temperature too low or PAD leak	Increase temperature; verify PAD not in sampling vocab
Infinite repetition in output	Greedy decoding or T too low	Use temperature sampling with $T \approx 0.8$
Slow training (>2 min/epoch)	CPU-only or large batch	Enable GPU; use <code>.to(device)</code> for model and data

Hands-on notebook: `Session_3_Shakespeare_Text_Generator.ipynb`

Instructor: Priyansh Saxena